Renewal Risk Intelligence Engine – Complete Project Explanation

> This document explains the entire project from scratch – what it is, why it exists, how every piece works – using simple analogies first, then the real implementation.

Table of Contents

---

1. What Is This Project?

One-Line Answer

> A tool that predicts which customers are about to cancel their subscription – and explains **why** – before the sales team even knows there's a problem.

Simple Explanation

Imagine you run Netflix. You have 120 company customers (not individuals – whole companies paying you $17K to $1.9M per year). Their contracts are coming up for renewal in the next 3 months.

The question: Which of these 120 companies are likely to **cancel** (churn) or **downgrade** their plan?

The old way: Sales people use gut feel, random Slack messages, and last-minute panic.

Our way: We build a system that:

1. Reads ALL the data about every customer (usage, complaints, survey scores, sales call notes, product changes)

2. Scores each customer from 0% to 100% risk
3. Labels them as **High / Medium / Low** risk
4. Writes a plain-English explanation: *"Acme Corp is high risk because their usage dropped 52%, they mentioned a competitor, and they're on an outdated software version"*
5. Discovers hidden patterns that no human would notice

---

2. The Real-World Problem

This project is built for **Contentstack** – a real company that sells a CMS (Content Management System) to other companies. Think of it like WordPress, but for enterprises.

The Business Scenario

The **BizOps team** (Business Operations) sits between Sales, Customer Success, Finance, and Product. Every quarter, they scramble to figure out:

- Which accounts might **not renew** their contract?
- Which accounts might **downgrade** to a cheaper plan?
- Where should we focus our time to **save** the most revenue?

Currently, they rely on scattered Salesforce notes, last-minute Slack threads, and gut feeling.

This is bad because by the time they notice a problem, the customer has already decided to leave.

What We Build

A **Renewal Risk Intelligence Engine** that:

- Ingests 5 different data sources (structured + unstructured)
- Reconciles messy, inconsistent data
- Extracts risk signals using math + AI
- Produces an actionable risk report with explanations

---

3. Simple Analogy: The Doctor's Diagnosis

Think of our pipeline as a **doctor diagnosing a patient**.

```
Patient    = A customer account (like "Acme Corp")
Symptoms   = Data signals (usage dropping, complaints increasing)
Lab Tests  = Our signal extractors (6 different "tests")
Diagnosis  = Risk Score (High / Medium / Low)
Treatment  = Recommended actions ("schedule an executive call")
```

Doctor's Process	Our Pipeline	What Happens
Collect patient files from 5 different hospitals	**Data Ingestion**	Load CSVs, text files, changelog
Verify the patient's name (John Smith vs. Jon Smth – same person?)	**Name Reconciliation**	Fuzzy-match "BritePath" to "BrightPath Solutions"
Run 6 lab tests (blood pressure, X-ray, blood sugar, etc.)	**Signal Extraction**	Usage trends, ticket analysis, NPS, CSM notes, SDK risk, engagement
Combine test results into an overall health score	**Risk Scoring**	Weighted average -> 0-100% composite score -> High/Medium/Low
Write a diagnosis report explaining findings	**LLM Explanation**	GPT writes: "This account is at risk because..."
Spot unusual patterns	**Non-Obvious Insights**	"This customer gave NPS 8 but is secretly building a replacement"

---

4. The Data We Have (Our Patient Records)

We are given **5 data files + 1 product changelog**. Think of each as a different "department" providing information about the same 120 customers.

4.1 accounts.csv (120 rows) – "The Registration Desk"

The master list of all customers.

```
| account_id | account_name       | arr     | contract_end_date | plan_tier  |
|------------|--------------------|---------|-------------------|------------|
| 1000       | Acme Corp          | $17,000 | 2026-05-25        | Starter    |
| 1007       | Zenith Publishing  | $1.6M   | 2026-05-05        | Enterprise |
```

Key fields:

- **ARR** (Annual Recurring Revenue): How much they pay per year. Losing $1.6M hurts WAY more than $17K.
- **contract_end_date**: When their contract expires. If within 90 days, we need to act NOW.
- **plan_tier**: Starter -> Growth -> Scale -> Enterprise

4.2 usage_metrics.csv (720 rows) – "The Activity Monitor"

6 months of product usage for each account (120 accounts x 6 months = 720 rows).

```
| account_id | month   | api_calls | active_users | sdk_version |
|------------|---------|-----------|--------------|-------------|
| 1000       | 2025-10 | 4,703     | 2            | v3.2.0      |
| 1000       | 2026-03 | 2,246     | 1            | v3.2.0      |
```

Acme Corp's API calls dropped from 4,703 to 2,246 (52% decline!). **This is like a gym member who used to come 5 days/week but now comes once.**

4.3 support_tickets.csv (271 rows) – "The Complaint Department"

Every support ticket filed.

- **P1 tickets** = critical production issues (customer's website is broken)
- **Open status** = nobody has fixed it yet
- BrightPath Solutions (1001) has **13 tickets with 8 P1s** – that's a fire alarm

4.4 nps_responses.csv (98 rows) – "The Survey Results"

Net Promoter Score (0-10):

- **0-6** = Detractor (unhappy)
- **7-8** = Passive (okay)
- **9-10** = Promoter (happy)

Traps intentionally placed in this data:

- Account 1017's comment is in **Chinese**
- Account 1014's comment is in **French**
- Account 1003 has NPS=8 but CSM notes say they're secretly leaving!
- Account 1019 has NPS=3 but comment says "phenomenal" – contradicts the score

4.5 csm_notes.txt (27 notes) – "The Doctor's Handwritten Notes"

Messy, unstructured notes from Customer Success Managers. **THE MOST VALUABLE but THE MESSIEST:**

- Contains competitor names (Hygraph, Strapi, Sanity, Contentful, Kontent.ai)
- Reveals internal politics (VP of Engineering on call = escalation)
- Exposes hidden churn ("classic silent churn pattern")
- Different date formats, misspelled names, inconsistent separators

4.6 changelog.md – "The Product Update Log"

Critical hidden items:

- SDK v3.x is being KILLED (security patches end April 30, 2026)
- Breaking change in v4.2.0 (changes API response format)
- If a customer is on v3.x, they MUST migrate or their app becomes insecure

---

5. Pipeline Step 1: Data Ingestion

> **File:** `src/data_loader.py` | **Analogy:** Opening all the envelopes and reading the letters inside

```
Raw Files                          Structured Data
accounts.csv          --parse-->   DataFrame (120 rows, typed columns)
usage_metrics.csv     --parse-->   DataFrame (720 rows, numeric)
support_tickets.csv   --parse-->   DataFrame (271 rows, dates parsed)
nps_responses.csv     --parse-->   DataFrame (98 rows, scores as int)
csm_notes.txt         --regex-->   List of 27 CSMNote objects
changelog.md          --regex-->   List of 8 ChangelogEntry objects
```

The easy parts (CSVs)

Loading CSVs is straightforward with pandas – read file, convert types.

The hard part: Parsing CSM Notes

The notes have DIFFERENT formats for every entry:

```
"Mar 12 - Acme Corp. They're frustrated..."
"3/15 acct 1001 - BritePath Solutions (sic) call..."
"2026-03-20 | NovaTech Industries | James O. ..."
"march 25 -- meridian health -- priya ..."
```

We split on `---` separators, then use multiple regex patterns to extract account IDs and names from each format.

---

6. Pipeline Step 2: Name Reconciliation (Fuzzy Matching)

> **File:** `src/reconciler.py` | **Analogy:** A receptionist matching hospital transfer papers where the name is misspelled

The Problem

CSM notes have typos:

CSM Note Says	Real Name	Match Type
"BritePath Solutions"	BrightPath Solutions	Fuzzy (88%)
"Pinacle Media"	Pinnacle Media Group	Fuzzy (78%)
"Thunderbolt Moters"	Thunderbolt Motors	Fuzzy (85%)
"vanguard retail"	Vanguard Retail	Exact (case-insensitive)

The 3-Step Matching Strategy

```
Step 1: Does the note have an explicit account ID? ("acct 1001") --> DONE
Step 2: Does the name match exactly (case-insensitive)? --> DONE
Step 3: Fuzzy match using RapidFuzz (threshold >= 65%) --> DONE or UNMATCHED
```

Result: 26 out of 27 CSM notes successfully matched!

---

7. Pipeline Step 3: Signal Extraction (6 Detectors)

> **File:** `src/signal_extractor.py` | **Analogy:** Running 6 different medical tests

Each detector produces a **risk score from 0.0 to 1.0** (0 = no risk, 1 = certain churn).

Detector 1: Usage Decline (Weight: 25%)

> "Is the customer using our product less?"

Compare the average of the first 2 months vs. last 2 months:

```
Acme Corp: API calls dropped from avg 4,353 to avg 2,371 = -45.5% --> risk 0.45
Ironclad Security: API calls GREW from 189,589 to 209,834 = +10.7% --> risk 0.00
```

Detector 2: Support Tickets (Weight: 15%)

> "Is the customer drowning in unresolved complaints?"

Combines: P1/P2 ticket ratio, open/escalated ratio, avg resolution time, total volume.

```
BrightPath (1001): 13 tickets, 8 P1/P2, 6 unresolved --> risk 0.62
Mosaic Digital (1023): 3 tickets, all resolved quickly --> risk 0.15
```

Detector 3: NPS Score (Weight: 10%)

> "What did the customer say on their satisfaction survey?"

- Score 0-6 (Detractor) –> risk 0.80-1.00
- Score 7-8 (Passive) –> risk 0.30-0.45
- Score 9-10 (Promoter) –> risk 0.00-0.10

Also: Translates Chinese/French/Spanish comments using LLM, and detects score-sentiment mismatches.

Detector 4: CSM Sentiment (Weight: 25%) – LLM-Powered

> "What did the sales team learn from talking to the customer?"

We send each CSM note to GPT-4o-mini and ask it to extract:

- Sentiment (positive/neutral/negative)

- Risk score (0-1)
- Competitor mentions
- Champion status (active/at_risk/lost)
- Key concerns
- Recommended actions

Example: From the note "They want a 30% discount or they walk. Competitor POC with Kontent.ai is already underway." GPT returns: sentiment=negative, risk=0.90, competitors=["Kontent.ai"].

Why LLM and not regex? No regex can reliably understand "CRO was cc'd which is never a good sign" or "classic silent churn pattern."

Detector 5: SDK Deprecation Risk (Weight: 15%)

> "Is the customer's technical setup about to become obsolete?"

From the changelog, we know SDK v3.x is being killed. So:

```
v3.1.2 or v3.2.0 --> risk 0.85 (deprecated, maximum migration effort)
v4.0.0           --> risk 0.25 (needs upgrade for breaking change)
v4.1.0           --> risk 0.15 (affected by response envelope change)
v4.2.3 or v4.3.0 --> risk 0.00 (current, safe)
```

Detector 6: Engagement / Shelfware (Weight: 10%)

> "Is the customer actually using what they're paying for?"

Compare active users vs. expected for their plan tier:

```
Enterprise plan expects ~80 users. If they only have 32, that's 40% engagement --> risk 0.70
Starter plan expects ~3 users. If they have 3, that's 100% --> risk 0.05
```

---

8. Pipeline Step 4: Risk Scoring (The Final Verdict)

> **File:** `src/risk_scorer.py` | **Analogy:** A teacher calculating a final grade

The Formula

```
Final Risk = (Usage x 25%) + (Tickets x 15%) + (NPS x 10%)
           + (CSM x 25%) + (SDK x 15%) + (Engagement x 10%)
```

Risk Tiers

```
Score >= 65% --> HIGH RISK    (likely to churn/downgrade)
Score >= 40% --> MEDIUM RISK  (needs attention)
Score <  40% --> LOW RISK     (healthy)
```

Example: Acme Corp (1000)

```
Usage Decline:   0.45 x 25% = 0.113
Support Tickets: 0.52 x 15% = 0.078
NPS Score:       0.80 x 10% = 0.080
CSM Sentiment:   0.80 x 25% = 0.200
SDK Deprecation: 0.85 x 15% = 0.128
Engagement:      0.70 x 10% = 0.070
-------------------------------------
COMPOSITE SCORE: 0.669 (66.9%)
RISK TIER:       HIGH
TOP DRIVERS:     CSM Sentiment, SDK Deprecation, Usage Decline
```

Contrast: Ironclad Security (1018) – Healthy

```
Usage: 0.00, Tickets: 0.15, NPS: 0.00, CSM: 0.10, SDK: 0.00, Engagement: 0.05
COMPOSITE: 0.053 (5.3%) --> LOW RISK
CSM Notes: "Want to add 30 more seats and upgrade to Enterprise!"
```

---

9. Pipeline Step 5: LLM-Powered Explanations

> **File:** `src/llm_engine.py` | **Analogy:** A doctor writing a patient summary

Why Numbers Alone Aren't Enough

If we tell the BizOps team "Acme Corp: Risk = 66.9%", they'll ask "WHY? What should I DO?"

What We Do

For every High/Medium risk account, we send ALL signals to GPT-4o-mini and ask for a 3-5 sentence briefing.

Example output:

> "Acme Corp is HIGH RISK with renewal in 25 days. Their API usage has dropped 52% over 6 months and they've gone from 2 active users to just 1. CSM notes reveal they're actively evaluating Hygraph, and they're on deprecated SDK v3.2.0 which loses security patches in April. **Recommended:** (1) Schedule urgent executive meeting. (2) Offer dedicated SA to help SDK migration."

The 4 LLM Uses in Our Pipeline

#	Use	Why LLM?
1	Translate NPS comments (Chinese, French, Spanish)	Language detection + translation
2	Analyze CSM notes (sentiment, competitors, concerns)	Messy human writing needs understanding
3	Write risk explanations (3-5 sentence briefings)	Synthesize 6 signals into a story
4	Discover cross-account insights	Find patterns humans would miss

---

10. Pipeline Step 6: Non-Obvious Insights

> **File:** `src/insights.py` | **Analogy:** A detective connecting dots across crime scenes

Why This Matters

A rule-based system can only check what you pre-program. It CANNOT notice:

Insight 1: Silent Churn (NPS-Usage Divergence)

```
Meridian Health:
  NPS Score: 8 (looks fine!)
  CSM Notes: "Usage has cratered. They built a homegrown solution.
              The score reflects they like our PEOPLE, not our PRODUCT."
  Usage: API calls dropped 39%

A rule-based system says "NPS 8 = fine"
Our system catches: NPS 8 + usage declining 39% = SILENT CHURN
```

Insight 2: SDK Deprecation Cascade

```
Changelog says: "SDK v3.x loses security patches April 30"
Usage data shows: 9 accounts still on v3.x
Ticket data shows: These same accounts have the MOST tickets

The chain: Deprecation --> Migration difficulty --> Tickets --> Frustration --> Churn
A rule-based system checking each in isolation would miss this chain.
```

Insight 3: Champion Loss as Leading Indicator

```
Orion Education:
  Usage right now: STABLE
  NPS: 8 (Fine!)

  But CSM notes: "Their champion (Director of Content) is nervous
                  about her own role post-merger."

  Champion loss today predicts usage decline 2-3 months from now.
  Rules watching metrics would miss this completely.
```

Insight 4: NPS Score-Sentiment Contradictions

```
Atlas Financial: NPS=8 but comment says "execution has fallen off a cliff"
Summit Analytics: NPS=3 but comment says "phenomenal"

The raw score alone is unreliable -- we detect these mismatches.
```

---

11. End-to-End Walkthrough With a Real Account

Let's trace **Zenith Publishing (1007)** through the ENTIRE pipeline.

Raw Data

- ARR: $1,625,000 | Plan: Enterprise | Renews: May 5, 2026
- Usage: 207K -> 79K API calls (-62%), 74 -> 32 users (-57%)
- Tickets: 6 total, 2 P1, 3 Open
- NPS: No response (missing!)
- CSM Note: "They want 30% discount or they walk. Competitor POC with Kontent.ai underway. CRO cc'd on email = never good sign."
- SDK: v3.1.2 (DEPRECATED – oldest version!)

Pipeline Processing

```
STEP 1: Data loaded successfully
STEP 2: CSM note matched via "#1007" --> Zenith Publishing
STEP 3: Signal scores computed:
  - Usage Decline:    0.50 (massive drop)
  - Support Tickets:  0.45 (moderate issues)
  - NPS:              0.50 (no response = unknown)
  - CSM Sentiment:    0.90 (competitor POC, discount demand)
  - SDK Risk:         0.85 (deprecated v3.1.2)
  - Engagement:       0.70 (32/80 expected users)
STEP 4: Composite = 0.666 --> HIGH RISK
STEP 5: LLM writes explanation about $1.6M at stake
STEP 6: Triggers SDK Cascade + Champion At Risk insights
```

Final Output

```
ZENITH PUBLISHING (1007)
Risk Score:    66.6% --> HIGH RISK
ARR at Risk:   $1,625,000
Renews In:     5 days (!!)
SDK:           v3.1.2 (DEPRECATED)
Competitors:   Kontent.ai (active POC)
Top Drivers:   CSM Sentiment (90%), SDK Deprecation (85%), Engagement (70%)
Action:        Executive meeting within 48 hours + Performance Engineer for 50K-entry issue
```

---

12. How the Code Maps to the Pipeline

File-by-File Breakdown

```
config.py               --> Settings, weights, thresholds, paths
src/data_loader.py      --> STEP 1: Load + parse all 5 data sources + changelog
src/reconciler.py       --> STEP 2: Fuzzy-match CSM note names to accounts
src/signal_extractor.py --> STEP 3: 6 risk detectors (0.0-1.0 each)
src/risk_scorer.py      --> STEP 4: Weighted composite scoring + tier assignment
src/llm_engine.py       --> STEP 5: All GPT-4o-mini calls
src/insights.py         --> STEP 6: Non-obvious pattern detection
run_pipeline.py         --> CLI entry point (calls Steps 1-6 in order)
app.py                  --> Streamlit dashboard (interactive web UI)
```

Execution Timeline

```
Step 1: Load data ..................... <1 second
Step 2: Reconcile names ............. <1 second
Step 3a: Usage signals .............. <1 second
Step 3b: Ticket signals ............. <1 second
Step 3c: NPS signals ................ ~3 seconds (1 LLM call for translation)
Step 3d: CSM signals ................ ~30-45 seconds (26 LLM calls!)
Step 3e: SDK risk ................... <1 second
Step 3f: Engagement signals ........ <1 second
Step 4: Composite scoring .......... <1 second
Step 5: LLM explanations ........... ~20-40 seconds (1 LLM call per at-risk account)
Step 6: Insights .................... ~5-10 seconds (1 LLM call)
-----------------------------------------------------
TOTAL: ~60-90 seconds, cost ~$0.02-0.05 per run
```

Visual Pipeline Flow

```
[  CLI Tables + Streamlit + CSV + JSON     ]
[=========================================]
```

Quick Reference: Key Terminology

Term	Simple Meaning
ARR	Annual Recurring Revenue – how much a customer pays per year
NPS	Net Promoter Score – customer satisfaction survey (0-10)
CSM	Customer Success Manager – the person who talks to customers
SDK	Software Development Kit – the code library customers use
Churn	When a customer cancels their subscription
Downgrade	When a customer switches to a cheaper plan
Shelfware	Software that's paid for but barely used
Silent Churn	Customer is leaving but still gives positive survey scores
Champion	The person inside a customer company who advocates for your product
P1 Ticket	Critical support ticket – production is broken
POC	Proof of Concept – testing a competitor's product
Composite Score	The final weighted risk score combining all 6 signals
Fuzzy Matching	Matching similar-but-not-identical text ("Pinacle" to "Pinnacle")
Signal	A single risk indicator from one data source
Detector	A function that computes a signal score from raw data
Pipeline	The end-to-end process from raw data to final output

---

End of Explanation Document